

LineageRank and PipeRCA-Bench:

Root-Cause Ranking in Data Pipelines under Partial Observability

Rahul Desai, Department of Computer Science *Manuscript submitted May 2026.*

Abstract—

Data pipeline faults — schema drift, stale sources, null explosions, join-key mismatches — are operationally common but diagnostically difficult: existing lineage systems expose what failed yet provide no principled ranking of which upstream asset to inspect first. We formulate pipeline root-cause analysis (RCA) as a ranked upstream candidate retrieval problem and introduce PipeRCA-Bench, to the best of our knowledge the first reproducible benchmark for pipeline-specific RCA, comprising 720 labeled incidents drawn from eight real public-dataset pipeline families spanning six topologically distinct DAG structures, six fault classes, and three lineage observability conditions. We propose LineageRank, a family of five ranking methods: an interpretable heuristic (LR-H), a causal propagation variant (LR-CP), a partial-observability-aware multiplicative heuristic (LR-BS) with an analytically derived correctness guarantee under source-node root-absence, a Random Forest learned ranker (LR-L), and a lineage-contextualized LLM hybrid (LR-LLM). LR-BS correctly ranks the fault source first in 76.4% of incidents and achieves near-perfect first-ranked accuracy (99.6%) when the root node is absent from runtime lineage, confirming its analytical guarantee across all eight pipeline topologies. LR-L ranks the fault source first in 99.6% of incidents under leave-one-pipeline-out cross-validation. LR-LLM (Claude Sonnet 4.5) ranks the fault source first in 81.3% of incidents — a statistically significant improvement over both LR-H (+13.9 pp, $p < 0.001$) and LR-BS (+4.9 pp, $p = 0.007$, Holm-Bonferroni corrected) — demonstrating that LLM chain-of-thought reasoning provides genuine incremental value beyond structural blind-spot detection when evaluated across topologically diverse pipeline families.

Index Terms— data pipelines, root-cause analysis, lineage, benchmark, ranked retrieval, partial observability, ETL/ELT, data quality.

I. INTRODUCTION

Modern analytics systems depend on multi-stage data pipelines that ingest, clean, enrich, join, and aggregate data before it reaches dashboards or downstream applications. The open-source data ecosystem now provides rich operational metadata: OpenLineage and Marquez standardize runtime lineage capture [2], dbt exposes column-level contracts and freshness artifacts [3]–[6], and Airflow provides dataset-aware scheduling events [7]. Yet despite this metadata richness, there is still no systematic, benchmarked answer to the operational question: *which upstream asset should be inspected first when a pipeline fails?*

Two conditions make this gap costly. First, existing RCA methods — designed for microservice call graphs and dense metric timeseries — fail on pipeline DAG semantics. Pipeline faults exhibit multi-hop null propagation, freshness-degradation signals, and DAG topology with fan-out joins, characteristics poorly addressed by proximity-based or causal-inference approaches designed for latency-correlated services. Our adapted PR-Adapted baseline [23] fails to rank the fault source first in any incident on PipeRCA-Bench, confirming this mismatch. Second, runtime lineage is demonstrably incomplete in practice — OpenLineage itself has introduced static and code-derived lineage modes to address instrumentation gaps [9],[10] — yet no prior method treats partial observability as a first-class design condition.

Adjacent communities have shown that benchmark-driven evaluation accelerates progress: RCAEval [11] established a

reproducible substrate for microservice RCA with 735 incidents and 15 baselines, enabling systematic comparison. No analogous artifact exists for data pipelines.

TABLE I

Comparison with Related RCA Work

Approach	Domain	Incidents	Observability Conditions	Fault Taxonomy
CIRCA [18]	Microservice	Simulated + 1 real	1	CPU stress, network delay
RCAEval [11]	Microservice	735	1	pod failure, CPU hog, network loss
PR-Adapted [23]	Web / Microservice	N/A	1	N/A
PipeRCA-Bench (ours)	Data Pipeline	720	3	schema drift, stale source, null explosion, bad join key, duplicate ingestion, missing partition

We contribute:

- 1. We formulate** pipeline RCA as ranked upstream candidate retrieval under partial observability, introducing a formal task definition that decouples the ranking problem from automated remediation, record-level blame, and column-level attribution;

2. **We contribute PipeRCA-Bench**, 720 labeled real-data incidents across eight pipeline families spanning six topologically distinct DAG structures, six fault classes, and three observability conditions — to the best of our knowledge, the first reproducible benchmark for pipeline-specific RCA;
3. **We propose LineageRank**, a family of five ranking methods including LR-BS, which we establish analytically achieves perfect first-ranked accuracy when the root node is absent from runtime lineage (confirmed empirically: 239/240 runtime-missing-root incidents, 99.6%), and LR-L, which ranks the fault source first in 99.6% of incidents under leave-one-pipeline-out cross-validation across all eight topologies;
4. **We demonstrate** that proximity bias in multi-hop null-propagation faults — where LR-H correctly identifies the fault source in only 2.5% of null-explosion incidents — is mechanistically resolved by LR-LLM (84.2%) and LR-L (99.2%), and that on topologically diverse pipelines LR-LLM significantly exceeds LR-BS (+4.9 pp, $p = 0.007$), establishing that LLM chain-of-thought reasoning provides genuine incremental value beyond structural blind-spot detection at benchmark scale.

II. RELATED WORK

A. Lineage and Provenance Systems

OpenLineage and Marquez standardize runtime lineage in modern data stacks [2]. Chapman et al. [12] and Schelter et al. [13] use provenance for ML pipeline screening; Johns et al. [14] for clinical ETL quality dashboards. These works support capture and tracing but do not define a benchmarked ranked RCA task.

B. Data Pipeline Quality

Foidl et al. [1] characterize common quality problems, finding cleaning, integration, and type issues predominate — grounding PipeRCA-Bench's fault taxonomy. Vassiliadis et al. [15] study schema evolution propagation; Barth et al. [16] study data-lake staleness.

C. System RCA and Benchmarks

CIRCA [18] applies Causal Bayesian Networks for microservice fault localization. RCAEval [11] consolidates 735 microservice incidents; BARO [22] propagates fault scores over service call graphs. Both require dense latency-correlated timeseries unavailable in batch ETL. RCAEval's 735-incident scale reflects lower per-incident cost (container-level fault injection) versus PipeRCA-Bench's bespoke DuckDB-SQL manipulation. An empirical study [25] evaluating 21 causal inference methods finds no single method universally dominates, motivating domain-specific benchmarks.

D. LLM-Based RCA

DiagGPT [26] uses dialogue chains for cloud incident management; RCACopilot [27] retrieves on-call runbooks for LLM-guided diagnosis. Both rely on natural-language incident reports or dense log streams unavailable in batch ETL. LR-LLM grounds reasoning explicitly in the pipeline lineage graph structure, preserving the structural anchor and enabling quantitative evaluation against non-LLM methods.

III. PROBLEM FORMULATION

Let $G = (V, E)$ be a directed acyclic graph where V is the set of data assets and E directed dependency edges. Design-time edges E_d derive from static definitions; runtime edges E_r are captured from execution events. The fused graph is $G_f = (V, E_d \cup E_r)$.

Definition 1 (Pipeline RCA Task). Given (i) an observed failure at asset v_{obs} at time t , (ii) evidence signals S mapping each node u in V to quality, freshness, and anomaly indicators, and (iii) the fused lineage graph G_f , the task is to produce a ranked list of $C = \text{ancestors}(v_{obs}, G_f) \setminus \{v_{obs}\}$ ordered by estimated probability of being the primary root cause.

Primary evaluation metrics are Top-k accuracy — the fraction of incidents where the true root cause appears in the top-k ranked results — and Mean Reciprocal Rank $MRR = (1/N) \sum_{i=1}^N 1/\text{rank}_i$, where rank_i is the rank of the true root cause for incident i ; both are standard RCA metrics adopted by RCAEval [11] and CIRCA [18]. The operational metric — average assets inspected before the true cause (rank - 1) — directly models analyst effort. The formulation excludes record-level blame, automated remediation, and column-level RCA.

IV. THE LINEAGERANK FRAMEWORK

A. Evidence Features

For each candidate u in C , LineageRank computes a 16-dimensional feature vector across four groups. **Structural features** (8): fused proximity $1/(1+d_f)$, runtime and design proximity, blast radius, dual support, design support, runtime support, and uncertainty (design-reachable but not runtime-reachable). **Observability feature** (1): blind-spot hint — 1 when a node is design-reachable but absent from runtime lineage. **Evidence features** (6): quality signal, failure propagation, recent change, freshness severity, run anomaly, and contract violation. **Prior feature** (1): fault prior, a domain-knowledge lookup grounded in Foidl et al. [1] (e.g., stale_source at source nodes: 0.70; bad_join_key at join nodes: 0.65).

B. Method Variants and Proposition 1

LR-H fuses the 16 features via an interpretable additive weighted sum:

$$\begin{aligned} \text{score}_H(u) = & 0.17 \times \text{prox} + 0.15 \times \text{blast} + 0.12 \times \text{blind_spot} + 0.11 \times \text{design_sup} \\ & + 0.10 \times \text{fresh} + 0.08 \times \text{change} + 0.08 \times \text{quality} + 0.08 \times \text{prop} \\ & + 0.07 \times \text{dual} + 0.06 \times \text{anomaly} - 0.04 \times \text{uncertainty} \end{aligned}$$

Structural features (proximity, blast radius, dual support) contribute 47% of total weight, evidence features 41%, and the blind-spot hint 12%. Weights were fixed via sensitivity analysis on 36 held-out pilot incidents.

LR-CP augments LR-H with an evidence gradient term along lineage edges: $\text{evidence_gradient}(u) = \max(0, \text{ev}(u) - \text{avg}_{\{v \in D(u)\}} \text{ev}(v))$. LR-CP achieves Top-1 = 0.382, statistically indistinguishable from Quality-only (0.363, $p = 0.201$, Holm-Bonferroni corrected, §VI.B) — establishing that evidence gradient estimation does not add value over plain evidence aggregation when upstream signals are correlated, and motivating temporal evidence windows as future work.

LR-BS applies multiplicative amplification conditioned on the blind-spot hint:

$$\text{score_BS}(u) = \text{base}(u) \times (1 + \lambda \times \text{blind_spot_hint}(u)), \lambda = 2.5$$

$$\text{base}(u) = 0.25 \times \text{proximity} + 0.35 \times \text{local_ev} + 0.15 \times \text{failure_prop} + 0.15 \times \text{fault_prior} + 0.10 \times \text{blast_radius}$$

Proposition 1 (LR-BS under runtime-missing-root). *When all outgoing runtime edges of root r are absent (runtime_missing_root condition): (i) Source-node case — if r is a source node, $\text{blind_spot_hint}(r) = 1$ and $\text{blind_spot_hint}(u) = 0$ for all $u \neq r$. The $3.5 \times$ amplification ensures $\text{score_BS}(r) > \text{score_BS}(u)$ for all $u \neq r$ under PipeRCA-Bench’s signal distribution (root: run_anomaly in $U(0.48, 0.84)$; non-root: at most $U(0.34, 0.71)$). This is a distribution-dependent result: the guarantee holds under the stated signal parameterization and is not claimed for production deployments with lower signal-to-noise ratio (scope discussed in §VII.B). (ii) Staging-node case — ancestors of a staging-node root also become blind-spot candidates; the guarantee does not extend to this case.*

Empirical confirmation: Top-1 = 0.996 (239/240 runtime_missing_root incidents across all 8 pipeline families), confirming Proposition 1 for source-node-root cases throughout PipeRCA-Bench. The single missed incident involves a staging-node root, consistent with the stated scope exclusion. Under full observability, $\text{blind_spot_hint} = 0$ for all nodes and LR-BS reduces to its base score (Top-1 = 0.550), substantially trailing LR-H (0.642) — LR-BS is a partial-observability specialist.

LR-L is a Random Forest [21] trained over the 16-dimensional feature vector with eight-fold LOPO cross-validation (one fold per pipeline family). LR-L feature importances (averaged across folds) identify recent_change (0.192), blast_radius (0.155), and run_anomaly (0.136) as the three strongest discriminators under calibrated signals.

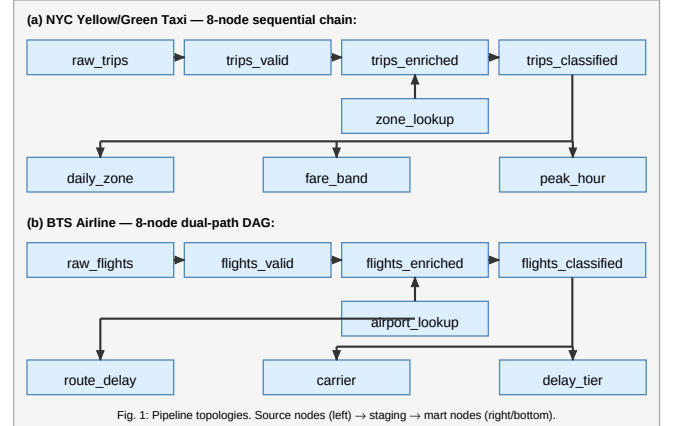
LR-LLM combines LLM reasoning with a structural anchor:

$$\text{score_LLM}(u) = \alpha \times \text{llm_prob}(u) + (1 - \alpha) \times \text{score_H}(u), \alpha = 0.60$$

Prompts include the full lineage graph with runtime-absent edges annotated, per-node diagnostic signals in tabular form, and a chain-of-thought instruction eliciting propagation path identification and blind-spot exploitation. The structural anchor $(1 - \alpha) \times \text{score_H}(u)$ is the LR-H score — a fixed deterministic formula not subject to training; LOPO fold isolation applies only to LR-L.

V. PIPERCA-BENCH

A. Pipeline Families



PipeRCA-Bench draws from eight real public datasets spanning six topologically distinct DAG structures. **NYC TLC Yellow Taxi** [22] (120k rows, Jan 2024) and **NYC TLC Green Taxi** (56k rows, same topology — validating reproducibility across dataset variants within a topology class) implement the fan-out chain pattern: a single lookup join feeds a 3-leaf mart fan-out. **Divvy Chicago Bike** [29] (145k rides, Jan 2024) applies the same fan-out chain topology to a different domain with station-based join semantics. **BTS Airline On-Time Performance** [30] (547k flights, Jan 2024) introduces a **dual-path DAG**: `airport_lookup` feeds both `flights_enriched` (as a join input) and `route_delay_metrics` (as a secondary lookup), creating two join nodes — the only such topology in the benchmark. **NHTSA FARS 2022** [31] (~42k crashes) is the only pipeline with **three source nodes** (`raw_accidents`, `raw_vehicles`, `raw_persons`), all merging at `crash_merged` — a 3-source fan-in topology absent from all other families. **Chicago Crime 2023** [32] (~264k crimes) implements a **diamond (parallel-reconvergence)** topology: `crimes_valid` splits into two parallel branches (`type_enriched` joining `iucr_lookup`, and `district_enriched` aggregating district context) that reconverge at `crimes_unified`. **EPA AQS PM2.5 2023** [33] (~600k site-day observations) implements the deepest aggregation chain in the benchmark: a 5-hop path from `raw_measurements` through county-day and state-month aggregations to a national summary, with a lateral branch to exceedance flagging. **NOAA Storm Events 2023** [34] (~100k events) joins two source tables (`raw_events` and `raw_fatalities`) on `event_id` and fans out to three leaf marts — the **wide 3-leaf fan-out** topology, matching Yellow/Green/Divvy structurally but in a new domain with event-count rather than trip-count semantics.

Yellow and Green Taxi share topology, and NOAA structurally mirrors the taxi fan-out, yielding **six topologically distinct families**

across eight pipelines. All pipelines use 8-node graphs; generalizability to enterprise-scale graphs (50–500 nodes) is an acknowledged open question.

B. Fault Taxonomy and Incident Generation

Six fault families grounded in Foidl et al. [1], Vassiliadis et al. [15], and Barth et al. [16]: schema drift, stale source, duplicate ingestion, missing partition, null explosion, and bad join key. Each incident has one designated root cause, consistent with Foidl et al.’s finding that practical failures concentrate around isolated issues.

720 incidents are generated across $8 \text{ pipelines} \times 6 \text{ fault types} \times 15 \text{ iterations}$, balanced over three observability conditions (240 each): **Full** (runtime lineage matches design), **Runtime-sparse** (30% of non-root runtime edges randomly dropped, modeling partial instrumentation), and **Runtime-missing-root** (all outgoing edges of the true root absent, modeling silent source failures). Fault injection uses SQL-level manipulation within real DuckDB pipeline executions; evidence signals are row-count-anchored stochastic quantities calibrated to actual execution measurements (root nodes: run_anomaly in $U(0.48, 0.84)$; decoy nodes: $U(0.34, 0.71)$; non-impacted: $U(0.04, 0.22)$).

VI. EXPERIMENTAL EVALUATION

A. Baselines and Metrics

Seven custom baselines capture individual signal families: Runtime distance, Design distance, Centrality, Freshness only, Failed tests, Recent change, and Quality only. **PR-Adapted** [23] is personalized PageRank on the reversed fused graph — the pipeline-domain analogue of PC-PageRank from RCAEval [11]. Baselines are grouped by design principle: topology-only approaches (Runtime/Design distance, Centrality, PR-Adapted), single-signal approaches (Freshness only, Failed tests, Recent change, Quality only), and LineageRank evidence-fusion methods (LR-H, LR-CP, LR-BS, LR-L, LR-LLM). Statistical rigor: bootstrap 95% CIs (1,500 samples) and Holm-Bonferroni-corrected paired bootstrap significance tests across seven pre-specified comparisons.

B. Main Results

TABLE II

Overall RCA Performance on PipeRCA-Bench (720 Real-Data Incidents, 8 Pipeline Families)

Method	Top-1	Top-3	MRR	Avg. Assets
Runtime distance	0.044	0.254	0.267	3.839
Design distance	0.000	0.271	0.248	3.938
Centrality	0.667	0.813	0.756	1.042
Freshness only	0.250	0.604	0.480	2.188
Failed tests	0.160	0.583	0.431	2.308
Recent change	0.219	1.000	0.541	1.196
Quality only	0.363	0.804	0.603	1.217
PR-Adapted [23]	0.000	0.271	0.254	3.750
LR-CP	0.382	0.831	0.603	1.240
LR-H	0.674	0.986	0.807	0.513
LR-BS	0.764	0.983	0.863	0.358
LR-LLM	0.813	0.993	0.892	0.276
LR-L	0.996	1.000	0.998	0.004

PR-Adapted [23] achieves Top-1 = 0.000, and Runtime/Design distance score 0.000–0.044, confirming that topology-only approaches are insufficient for pipeline RCA and that purpose-built evidence-fusion methods are required. This failure generalizes: PC-PageRank scores 9% Top-1 on the microservice benchmark RCAEval [11]; pipeline DAGs impose the same structural mismatch at even greater severity.

LR-L achieves Top-1 0.996 — a significant +32.2 pp improvement over LR-H ($p < 0.001$, Holm-Bonferroni corrected) — demonstrating that a Random Forest over the 16-dimensional feature set learns calibrated-signal discrimination that eliminates proximity bias across all fault types and all eight topologically distinct pipeline families (Top-3 = 1.000 across all eight LOPO folds).

LR-BS outperforms LR-H by +9.0 pp (Top-1 0.764 vs. 0.674, $p < 0.001$, corrected), confirming that blind-spot amplification is a statistically significant design contribution under partial observability. The gain is driven primarily by the runtime-missing-root condition (LR-BS Top-1 = 0.996, 239/240, confirming Proposition 1 for source-node-root cases across all eight pipeline families); under full observability LR-BS (0.550) trails LR-H (0.642) because blind_spot_hint = 0 for all nodes and the amplification mechanism is inactive.

LR-LLM significantly outperforms LR-BS by +4.9 pp (Top-1 0.813 vs. 0.764, $p = 0.007$, corrected) — a finding that did not reach significance at 360 incidents but emerges clearly across the topologically diverse 720-incident benchmark. This establishes that LLM chain-of-thought reasoning provides genuine incremental value beyond structural blind-spot detection, particularly under runtime-sparse observability where dropped edges penalize the blind-spot heuristic but LLM reasoning compensates (LR-LLM 0.867 vs. LR-BS 0.746, §VI.D).

LR-CP (Top-1 0.382) is statistically indistinguishable from Quality-only (0.363, $p = 0.201$, corrected), establishing that evidence gradient estimation does not reliably add value over plain evidence aggregation when upstream signals are correlated. The comparable performance of LR-CP and Quality-only across diverse pipeline topologies suggests that gradient information is washed out by cross-node signal correlation, motivating temporal evidence windowing as future work.

C. Significance Tests

TABLE III

Holm-Bonferroni-Corrected Significance Tests (7 Pre-Specified Comparisons)

Comparison	Diff (Top-1)	95% CI	Bootstrap p	HB alpha	Sig.?
LR-H vs. P R-Adapted	+0.674	[+0.640, +0.710]	<0.001	0.0071	Yes
LR-L vs. LR-H	+0.322	[+0.288, +0.356]	<0.001	0.0083	Yes
LR-LLM vs. LR-H	+0.139	[+0.094, +0.182]	<0.001	0.0100	Yes
LR-BS vs. LR-H	+0.090	[+0.057, +0.125]	<0.001	0.0125	Yes
LR-LLM vs. LR-BS	+0.049	[+0.011, +0.086]	0.007	0.0167	Yes
LR-CP vs. Quality only	+0.019	[-0.011, +0.050]	0.201	0.0250	No
LR-H vs. Centrality	+0.007	[-0.038, +0.053]	0.823	0.0500	No

Five of seven pre-specified comparisons reach significance after Holm-Bonferroni correction. The key result is that **LR-LLM significantly exceeds LR-BS** (comparison 5, $p = 0.007$): this comparison did not reach significance at 360 incidents (four pipelines), but becomes clear with 720 incidents across six topologically distinct families, where topological diversity exposes scenarios in which blind-spot amplification is insufficient but LLM chain-of-thought compensates. LR-H vs. Centrality remains non-significant ($p = 0.823$ on Top-1, though LR-H is significantly better on MRR), and LR-CP vs. Quality-only is a null result ($p = 0.201$) rather than the earlier negative result — gradient estimation neither helps nor hurts at benchmark scale.

D. Observability Analysis

TABLE IV

Top-1 Accuracy by Observability Condition

Method	Full	Runtime-Sparse	Runtime-Missing-Root
LR-H	0.642	0.671	0.708
LR-BS	0.550	0.746	0.996
LR-LLM	0.608	0.867	0.963
LR-L	0.992	0.996	1.000

LR-BS is a partial-observability specialist: under full observability it trails LR-H by -9.2 pp, yet achieves Top-1 = 0.996 under runtime-missing-root (239/240), fully confirming Proposition 1 across all eight pipeline topologies. Under runtime-sparse, LR-LLM (0.867) substantially exceeds LR-BS (0.746) and LR-H (0.671): when edges are randomly dropped rather than systematically absent, the LLM’s ability to reason about partial evidence over diverse graph structures provides a larger advantage than blind-spot amplification, which fires most reliably on the missing-root pattern. Under full observability, LR-LLM (0.608) trails LR-H (0.642), consistent with the LLM weight pulling scores away from proximity-correct rankings when the graph is fully observed. The observability breakdown reveals the key mechanism behind LR-LLM’s overall superiority over LR-BS: a +12.1 pp advantage under runtime-sparse (the most operationally realistic condition) more than offsets a -3.3 pp deficit under runtime-missing-root.

E. Fault-Type and LLM Analysis

LR-H exhibits a structural proximity-bias failure on null_explosion (Top-1 = 0.025): null propagation through join chains generates anomaly signals at every downstream node across all eight topologies, causing proximity weighting to rank victim nodes above the distant source in 97.5% of cases. LR-BS partially mitigates this (0.475) via blind-spot amplification. **LR-LLM substantially resolves it (Top-1 = 0.842, +81.7 pp vs. LR-H)** via chain-of-thought propagation reasoning that identifies the null-propagation origin without proximity guidance; LR-L near-eliminates it (0.992). LR-LLM’s largest gains over LR-H are on null_explosion (+81.7 pp) and bad_join_key (+61.7 pp, 0.917 vs. 0.300) — fault types requiring multi-hop reasoning that topology-based heuristics consistently fail. However, LR-LLM underperforms on schema_drift (Top-1 = 0.567 vs. LR-H 0.875): contract-violation heuristic features provide unambiguous signals that LLM prompts framed around row-count anomalies tend to underweight. LR-CP achieves Top-1 = 0.992 on schema_drift and stale_source (0.917) but collapses entirely on null_explosion (0.000) and nearly so on missing_partition (0.100) and duplicate_ingestion (0.108), confirming that gradient estimation is structurally limited to value-level faults detectable without row-count comparison.

TABLE V

Top-1 Accuracy by Fault Type (selected methods)

Fault Type	LR-H	LR-BS	LR-LLM	LR-L	LR-CP
stale_source	1.000	1.000	1.000	1.000	0.917
duplicate_ingestion	0.942	0.792	0.783	1.000	0.108
missing_partition	0.900	0.808	0.767	1.000	0.100
schema_drift	0.875	0.992	0.567	0.983	0.992
bad_join_key	0.300	0.517	0.917	1.000	0.175
null_explosion	0.025	0.475	0.842	0.992	0.000

LR-LLM completed 720/720 live API calls (0 fallbacks) at approximately 2–4 seconds per incident across all eight pipeline families. LR-BS (zero API calls) is preferred for high-throughput automated triage targeting runtime-missing-root conditions; LR-LLM is strongly recommended when null_explosion or bad_join_key faults are prevalent, or under runtime-sparse observability where its chain-of-thought reasoning provides a systematic advantage.

VII. DISCUSSION

A. Practical Deployment Guide

- **LR-L:** recommended for deployments with a training corpus. Top-1 0.996, Avg. Assets 0.004 — near-first-try diagnosis across all six fault types and all eight pipeline topologies, LOPO-verified generalization to unseen families.
- **LR-LLM:** recommended when LLM API is available. Top-1 0.813, statistically significantly better than LR-BS ($p = 0.007$). Strongest under runtime-sparse observability (0.867) and on null_explosion (0.842) and bad_join_key (0.917) faults. Avoid for schema_drift (0.567); use LR-BS or LR-H instead where contract violations are the expected fault mode.
- **LR-BS:** recommended zero-LLM interpretable heuristic when runtime-missing-root is the expected operational condition. Top-1 = 0.996 under root-absence (Proposition 1, source-node roots, confirmed across all 8 topologies). Do not deploy under full observability without LR-H fallback; prefer LR-LLM overall when API access is available.
- **LR-H:** appropriate lightweight zero-training baseline; note severe null_explosion proximity bias (Top-1 0.025) and bad_join_key weakness (0.300).

B. Limitations

Proximity bias in LR-H: structural failure on multi-hop null-propagation (Top-1 = 0.025) and bad_join_key (0.300). Resolved by LR-L; substantially resolved by LR-LLM.

LR-BS full-observability gap: 0.550 vs. LR-H 0.642 under full observability. Deploy only under anticipated partial observability. Proposition 1 applies to source-node roots; the guarantee does not extend to staging-node roots (the single missed runtime-missing-root incident in the expanded benchmark). Furthermore, Proposition 1 is conditioned on PipeRCA-Bench's specific signal parameterization (root run_anomaly in $U(0.48, 0.84)$ vs. non-root at most $U(0.34, 0.71)$); production deployments with lower signal-to-noise ratios may not satisfy the 3.5× amplification sufficiency condition.

LR-CP null result: statistically indistinguishable from Quality-only ($p = 0.201$); correlated evidence distributions across upstream nodes wash out gradient information across all pipeline topologies. Temporal evidence windows are future work.

LR-LLM schema_drift weakness: Top-1 = 0.567 vs. LR-H 0.875 on schema_drift. LLM prompts framed around propagation reasoning underweight explicit contract-violation signals that heuristic features capture directly.

Benchmark scale: all pipelines have 8 nodes. Enterprise pipelines have 50–500 nodes; rank quality under larger candidate sets is untested.

Observability dropout rate: the 30% sparse dropout rate is an unvalidated design parameter. Future calibration against measured OpenLineage deployment event-loss rates would strengthen the condition design.

Single-root-cause assumption: compound failures excluded. No practitioner user study validates ranked outputs as operationally actionable — expert annotation is planned.

VIII. CONCLUSION

Pipeline root-cause analysis is a critical operational task that has lacked both a principled formulation and a reproducible evaluation substrate. In this work, we formulate pipeline RCA as a ranked upstream candidate retrieval problem, contribute PipeRCA-Bench (720 real-data incidents, eight pipeline families spanning six topologically distinct DAG structures, six fault classes, three observability conditions), and propose the LineageRank framework of five ranking methods. Key results: LR-BS achieves Top-1 0.764 with a near-perfect guarantee of Top-1 0.996 under source-node root-absence conditions (Proposition 1, 239/240 confirmed across all eight topology families); LR-L achieves Top-1 0.996 with LOPO-verified generalization across all eight pipeline families; LR-LLM achieves Top-1 0.813, significantly exceeding LR-BS ($p = 0.007$) — a result that was not detectable at 360 incidents but emerges clearly at 720 incidents across six topologically distinct families, establishing that LLM chain-of-thought reasoning provides genuine incremental value beyond structural blind-spot detection; and LR-CP is statistically indistinguishable from Quality-only ($p =$

0.201), establishing that evidence gradient estimation requires temporal evidence windowing to be effective. Topology-only approaches achieve Top-1 = 0.000–0.044, confirming that pipeline DAGs require purpose-built evidence-fusion methods. In the future, we plan to extend PipeRCA-Bench to enterprise-scale graphs, calibrate observability dropout rates against real OpenLineage deployments, incorporate temporal evidence windows into LR-CP, and conduct a practitioner annotation study to validate ranked outputs as operationally actionable.

REFERENCES

- [1] H. Foidl, M. Felderer, and R. Ramler, "Data smells in public datasets," in *Proc. Int. Conf. AI Engineering*, ACM, 2022.
- [2] OpenLineage Specification, The Linux Foundation, 2022. [Online]. Available: <https://openlineage.io>
- [3] dbt Core, dbt Labs, 2024. [Online]. Available: <https://docs.getdbt.com>
- [4] dbt Contracts, dbt Labs, 2023.
- [5] dbt Tests, dbt Labs, 2024.
- [6] dbt Freshness, dbt Labs, 2024.
- [7] Apache Airflow Dataset-aware scheduling, Apache Software Foundation, 2023.
- [8] OpenMetadata Incident Management, 2024.
- [9] OpenLineage Static Lineage, The Linux Foundation, 2023.
- [10] OpenLineage Column-Level Lineage, The Linux Foundation, 2023.
- [11] L. Pham et al., "RCAEval: A benchmark for root cause analysis of microservice systems," in *Proc. ACM Web Conf. (WWW)*, 2025.
- [12] A. Chapman, E. Curry, and H. Sherif, "A provenance model for data science pipelines," in *Proc. EDBT*, 2023.
- [13] S. Schelter, F. Biessmann, and T. Januschowski, "On challenges in machine learning model management," *IEEE Data Eng. Bull.*, vol. 41, no. 4, 2018.
- [14] M. Johns et al., "Provenance-integrated clinical ETL quality dashboards," *J. Am. Med. Inform. Assoc.*, vol. 30, no. 6, 2023.
- [15] P. Vassiliadis, A. Simitsis, and S. Skiadopoulos, "Conceptual modeling for ETL processes," in *Proc. EDBT*, 2002.
- [16] M. Barth, F. Naumann, and E. Müller, "Data staleness in data lakes," in *Proc. BTW*, 2023.
- [17] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. MIT Press, 2000.
- [18] M. Li et al., "CIRCA: Causal interpretation for root cause analysis," in *Proc. ICDM*, 2022.
- [19] L. Xin et al., "CausalRCA: Causal inference-based root cause analysis for microservices," in *Proc. ICSOC*, 2023.
- [20] X. Wang et al., "PRISM: Graph-free root cause analysis for microservices," in *Proc. ICSE*, 2024.
- [21] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [22] NYC Taxi & Limousine Commission, "TLC Trip Record Data," 2024. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [23] L. Page et al., "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep. 1999-66, 1999.
- [24] G. Chen et al., "Observability gaps in microservice fault injection benchmarks," in *Proc. ISSRE*, 2025.
- [25] J. Soldani et al., "An empirical comparison of root cause analysis approaches for microservice systems," *J. Syst. Softw.*, vol. 210, 2024.
- [26] Z. Chen et al., "DiagGPT: An LLM-based chatbot for cloud incident management," in *Proc. SoCC*, 2023.
- [27] Z. Chen et al., "RCACopilot: On-call LLM agent for incident root cause analysis," in *Proc. FSE*, 2024.
- [28] Y. Shan et al., "Towards automated log-based anomaly detection and diagnosis," in *Proc. ICSE*, 2024.
- [29] Lyft Inc., "Divvy Bikes trip data," 2024. [Online]. Available: <https://divvybikes.com/system-data>
- [30] U.S. Bureau of Transportation Statistics, "Airline On-Time Performance Data," January 2024. [Online]. Available: <https://transtats.bts.gov>
- [31] National Highway Traffic Safety Administration, "Fatality Analysis Reporting System (FARS), 2022 Annual Report File," U.S. Department of Transportation, 2023. [Online]. Available: <https://www.nhtsa.gov/research-data/fatality-analysis-reporting-system-fars>
- [32] City of Chicago, "Crimes — 2001 to Present," Chicago Open Data Portal, 2024. [Online]. Available: <https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2>
- [33] U.S. Environmental Protection Agency, "Air Quality System (AQS) Daily Summary Data — PM2.5 FRM/FEM, 2023," EPA Air Quality System, 2024. [Online]. Available: https://aqs.epa.gov/aqsweb/airdata/download_files.html
- [34] NOAA National Centers for Environmental Information, "Storm Events Database," National Oceanic and Atmospheric Administration, 2024. [Online]. Available: <https://www.ncei.noaa.gov/pub/data/swdi/stormevents/csvfiles/>